

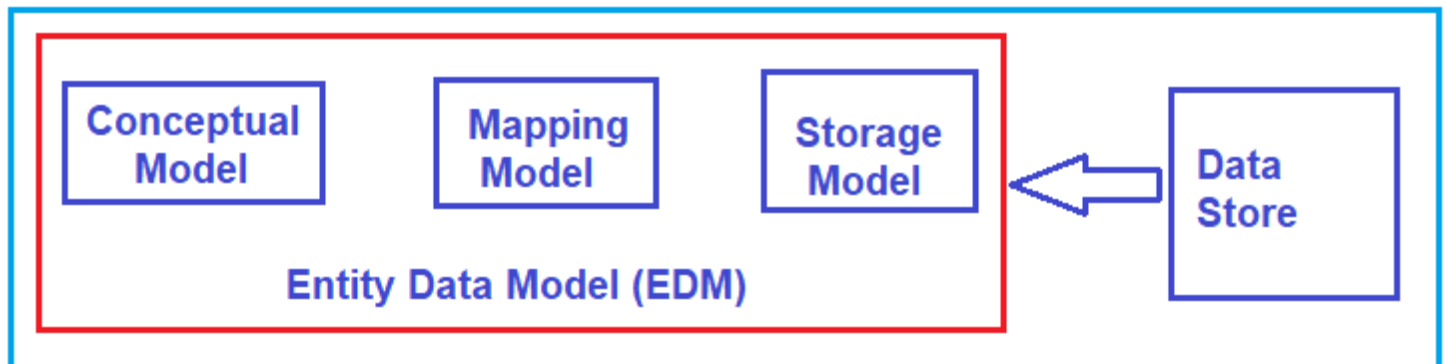
Entity Data Model (EDM) of Entity Framework Database First Approach

In this article, I am going to discuss the **Entity Data Model (EDM) of the Entity Framework Database First Approach** in detail. We are going to work with the same example that we created in [Our Previous Article](#). Please read our previous article before proceeding to this article where we created the required database tables, views, and stored procedure with test data as well as we also discussed the step-by-step procedure to interact with the database using [Entity Framework Database First Approach](#).

Entity Data Model (EDM) in Entity Framework:

Entity Framework uses EDM (Entity Data Model) for all Database-Related Operations. Entity Data Model is a model that describes the entities and the relationships between them. We need to consider 3 core parts that form the basis of the Entity Framework and collectively it is known as the Entity Data Model. The following are the three core parts of EDM.

1. **The Storage Schema Definition Model (SSDL)**
2. **The Conceptual Schema Definition Model (CSDL)**
3. **The Mapping Model (Conceptual – Storage Mapping (MSL))**



The Storage Schema Model in Entity Framework

The Storage Schema Model also called Storage Schema Definition Layer (SSDL) represents the schematic representation of the backend data store or in simple words, we can say it represents the XML view of the corresponding database. That means what are the database tables, views, stored procedures, what are primary keys, and the foreign key relationship between the database tables, that you have included while creating the EDMX file, all those information is going to represent by this Storage Schema Definition Layer (SSDL) Model. So, it is all about the backend database.

The Conceptual Schema Model in Entity Framework

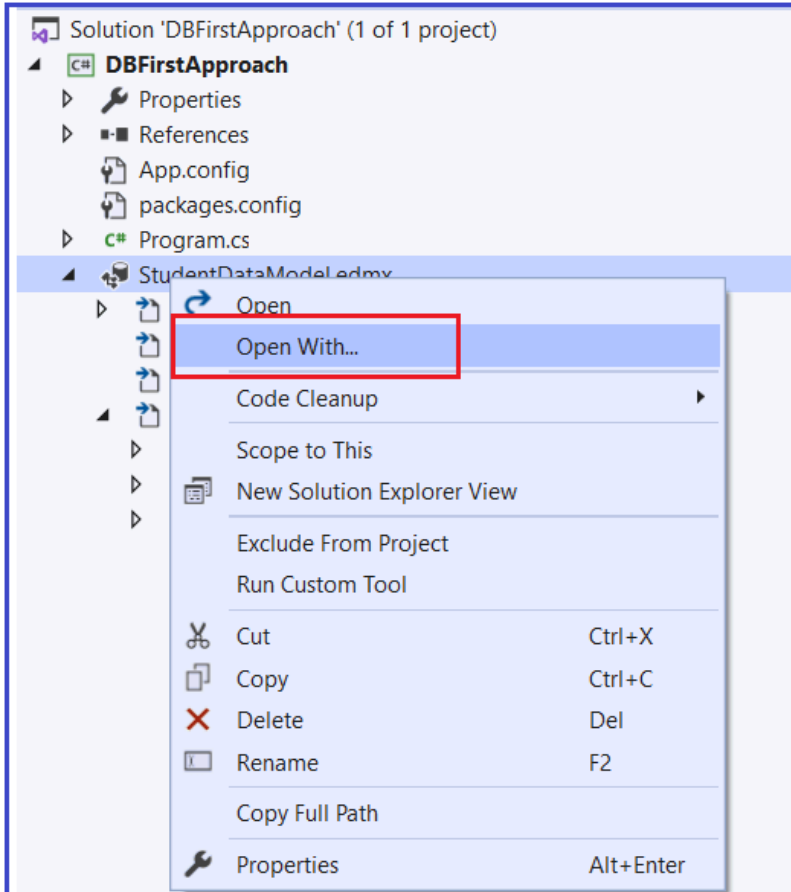
The Conceptual Schema Model also called as Conceptual Schema Definition Layer (CSDL) is the real entity model, and using this entity model we are going to write our queries. That means based on the database tables, views, stored procedures, primary key, and foreign keys relationship between the database tables, this Conceptual Schema Definition Layer (CSDL) creates the corresponding classes and functions. For database tables and views, it is going to create classes and for the stored procedures, it is going to create functions. For primary and foreign keys, it is going to create navigation properties inside the classes. And we are going to work with these classes and functions in Entity Framework.

The Mapping Model (Conceptual – Storage Mapping) in Entity Framework

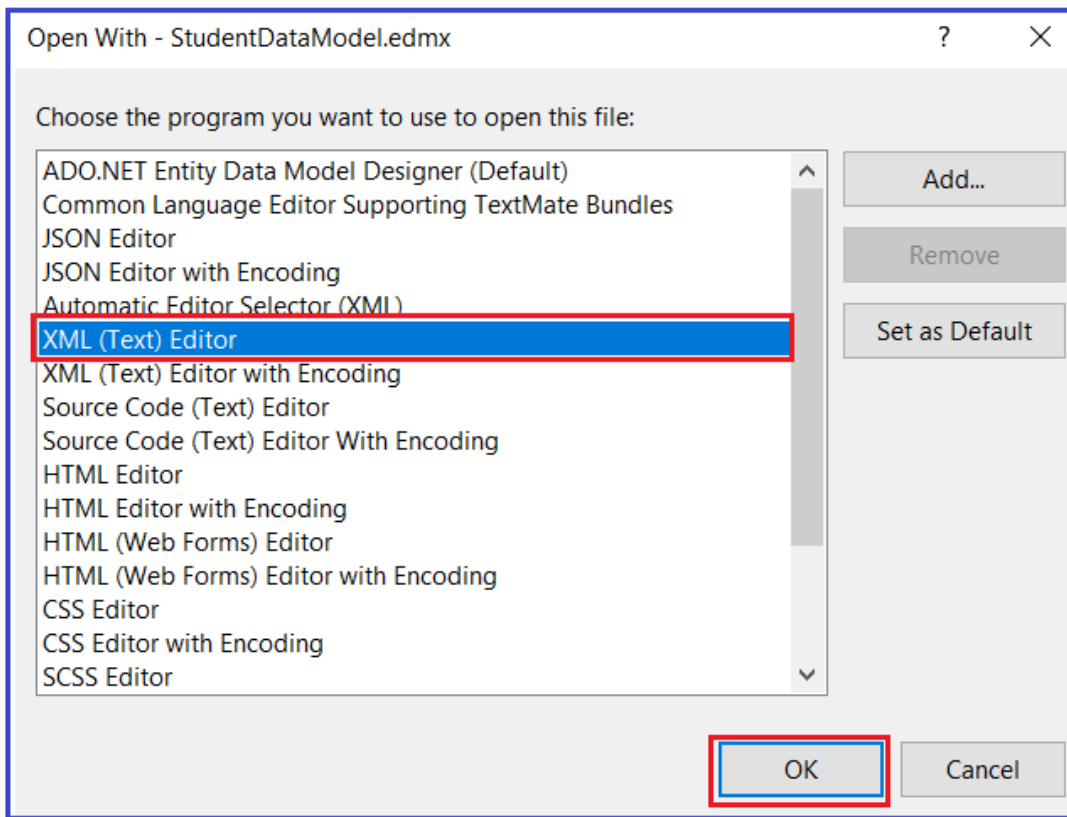
The Mapping Layer is just a mapping between the Conceptual Model and the Storage Model. So, the Storage Schema Definition Layer (SSDL) represents the backend database, and Conceptual Schema Definition Layer (CSDL) represents the corresponding classes and functions. But how entity framework will come to know which class is for which database table, and which property of a class is for which database table columns? We need to define this mapping information and Entity Framework defined this mapping information as this Mapping Model.

How to View the Schema Definition of EDM in Entity Framework?

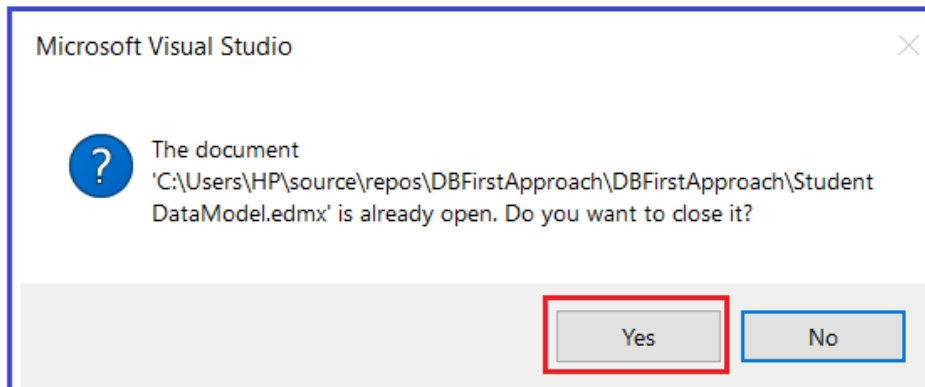
In order to see the three parts of the Entity Data Model i.e. **Conceptual Schema (CSDL)**, **Storage Schema (SSDL)**, and **Mapping Schema (MSL)**, you need to open the EDM (Entity Data Model of Entity Framework) designer in XML view. To do so, Right-click on **StudentDataModel.edmx** -> and then click on the **'Open with..'** option as shown below.



This will open the following popup window. Here, you need to select **'XML (text) Editor'** and click on the **OK** button as shown below.



The Visual Studio cannot display the Entity Data Model in both Design View and in XML Format at the same time. So, it will ask you with one popup to close the EDMX file if the Design View of EDM is opened, just click Yes as shown in the below image.



Once you click Yes, it will open the XML view of the Entity Data Model as shown in the below image. You can see the following XML view by toggling all outlining as shown below.

```

<?xml version="1.0" encoding="utf-8"?>
<edmx:Edmx Version="3.0" xmlns:edmx="http://schemas.microsoft.com/ado/2009/11/edmx">

  <!-- EF Runtime content -->
  <edmx:Runtime>
    <!-- SSDL content -->  Represents the Database Schemas in XML Format
    <edmx:StorageModels>...</edmx:StorageModels>

    <!-- CSDL content -->  Represents the Entities Based on the Database Schemas in XML Format
    <edmx:ConceptualModels>...</edmx:ConceptualModels>

    <!-- C-S mapping content -->  Represents the Mapping Between the Database Schema and Entities
    <edmx:Mappings>...</edmx:Mappings>
  </edmx:Runtime>

  <!-- EF Designer content (DO NOT EDIT MANUALLY BELOW HERE) -->
  <Designer xmlns="http://schemas.">...</Designer>  This Represents the EF Designer Content i.e. in the EDM Designer View how Entities are going to displayed
</edmx:Edmx>

```

You can see the SSDL content, the CSDL content, and the C-S Mapping Content here. If you expand the SSDL and CSDL, each one has some common XML node under each schema node. You don't need to edit the XML data because this can be accomplished easier in the Model Browser.

SSDL Content of Entity Data Model of Entity Framework:

The SSDL (Storage Schema Definition Layer) represents the Schematic Representation of the backend database. Now, expand the SSDL section as shown below and you can see each database table and views are represented by EntityType, the foreign keys are represented by Association, and the stored procedures are represented by Function. Here, the primary key inside an Entity Type is represented by the key node. Each column of a table is represented by a Property attribute and here, for the data type, you can see the database data type, not the .NET Data type as it represents the database schema. Whatever constraint you applied to the database columns like Identity, Nullable or not, everything is going to be represented by this SSDL Section of Entity Data Model in Entity Framework.

```

<!-- SSDL content -->
<edmx:StorageModels>
  <Schema Namespace="EF_Demo_DBModel.Store" Provider="System.Data.SqlClient" ProviderManifestToken="2012"
    Alias="Self" xmlns:store="http://schemas.microsoft.com/ado/2007/12/edm/EntityStoreSchemaGenerator"
    xmlns:customannotation="http://schemas.microsoft.com/ado/2013/11/edm/customannotation"
    xmlns="http://schemas.microsoft.com/ado/2009/11/edm/ssdl">
    <EntityType Name="Course">  Database Table Name
      <Key>
        <PropertyRef Name="CourseId" />  Primary Key
      </Key>
      <Property Name="CourseId" Type="int" StoreGeneratedPattern="Identity" Nullable="false" />
      <Property Name="CourseName" Type="varchar" MaxLength="100" />  Database Columns
      <Property Name="TeacherId" Type="int" />
    </EntityType>
    <EntityType Name="Standard">
      <Key>
        <PropertyRef Name="StandardId" />
      </Key>
      <Property Name="StandardId" Type="int" StoreGeneratedPattern="Identity" Nullable="false" />
      <Property Name="StandardName" Type="varchar" MaxLength="100" />
      <Property Name="Description" Type="varchar" MaxLength="100" />
    </EntityType>
  </Schema>
</edmx:StorageModels>

```

Note: It is using the Database DataTypes such as int, varchar, etc.

CSDL Content of Entity Data Model of Entity Framework:

The CSDL (Conceptual Schema Definition Layer) represents the Entities based on the corresponding database, and using this entity model we are going to write our queries. Now, expand the CSDL section as shown below and you can see each class is represented by EntityType, and the foreign keys are represented by Navigational Properties. Here, the primary key inside an Entity Type is represented by the key node. Each property of a class is represented by the Property attribute and here, for the data type, you can see the .NET data type as it represents the .NET classes. Whatever constraint you applied to the database columns; everything is going to be applied to the class properties by this CSDL Section of Entity Data Model in Entity Framework.

```
<!-- CSDL content -->
<edmx:ConceptualModels>
  <Schema Namespace="EF_Demo_DBModel" Alias="Self" annotation:UseStrongSpatialTypes="false"
    xmlns:annotation="http://schemas.microsoft.com/ado/2009/02/edm/annotation"
    xmlns:customannotation="http://schemas.microsoft.com/ado/2013/11/edm/customannotation"
    xmlns="http://schemas.microsoft.com/ado/2009/11/edm">
    <EntityType Name="Course"> ← Class Name Representing the Course Database table
      <Key>
        <PropertyRef Name="CourseId" /> ← Primary Key
      </Key>
      <Property Name="CourseId" Type="Int32" Nullable="false" annotation:StoreGeneratedPattern="Identity" />
      <Property Name="CourseName" Type="String" MaxLength="100" FixedLength="false" Unicode="false" />
      <Property Name="TeacherId" Type="Int32" /> ← Class Properties
      <NavigationProperty Name="Teacher" Relationship="Self.FK__Course__TeacherI__29572725" FromRole="Course" ToRole="Teacher" />
      <NavigationProperty Name="Students" Relationship="Self.StudentCourse" FromRole="Course" ToRole="Student" />
    </EntityType>
    <EntityType Name="Standard">
      <Key>
        <PropertyRef Name="StandardId" />
      </Key>
      <Property Name="StandardId" Type="Int32" Nullable="false" annotation:StoreGeneratedPattern="Identity" />
      <Property Name="StandardName" Type="String" MaxLength="100" FixedLength="false" Unicode="false" />
      <Property Name="Description" Type="String" MaxLength="100" FixedLength="false" Unicode="false" />
      <NavigationProperty Name="Students" Relationship="Self.FK__Student__Standar__2C3393D0" FromRole="Standard" ToRole="Student" />
      <NavigationProperty Name="Teachers" Relationship="Self.FK__Teacher__Standar__267ABA7A" FromRole="Standard" ToRole="Teacher" />
    </EntityType>
  </Schema>
</edmx:ConceptualModels>
```

↑ Foreign Keys Represented by Navigational Properties

C-S Mapping Content of Entity Data Model of Entity Framework:

C-S Mapping Content of EDM represents the mapping between the Conceptual Model and the Storage Model. That means for which database table which model class is to map. For the columns of a database table, which properties of a class are to map? Entity Framework uses this CS-Mapping Content to identify, whenever we are doing any operations using the Entities, which database table need to use behind the scene. Now, expand the C-S Mapping section and you will see the following content. It is showing the database table and the model class name and the column names mapped with the properties of the model class.

```

<!-- C-S mapping content -->
<edmx:Mappings>
  <Mapping Space="C-S" xmlns="http://schemas.microsoft.com/ado/2009/11/mapping/cs">
    <EntityContainerMapping StorageEntityContainer="EF_Demo_DBModelStoreContainer"
      CdmEntityContainer="EF_Demo_DBEntities">
      <EntitySetMapping Name="Courses"> ← Database Table Name
        <EntityTypeMapping TypeName="EF_Demo_DBModel.Course"> ← Model Class Name
          <MappingFragment StoreEntitySet="Course">
            <ScalarProperty Name="CourseId" ColumnName="CourseId" />
            <ScalarProperty Name="CourseName" ColumnName="CourseName" />
            <ScalarProperty Name="TeacherId" ColumnName="TeacherId" />
          </MappingFragment>
        </EntityTypeMapping>
      </EntitySetMapping>
      <EntitySetMapping Name="Standards">
        <EntityTypeMapping TypeName="EF_Demo_DBModel.Standard">
          <MappingFragment StoreEntitySet="Standard">
            <ScalarProperty Name="StandardId" ColumnName="StandardId" />
            <ScalarProperty Name="StandardName" ColumnName="StandardName" />
            <ScalarProperty Name="Description" ColumnName="Description" />
          </MappingFragment>
        </EntityTypeMapping>
      </EntitySetMapping>
    </EntityContainerMapping>
  </Mapping>

```

↑ Model Properties
↑ Database Column Names

Schema Definition Language in Entity Framework

Databases use Data Definition Language to define the structure of the database tables. Similarly, ADO.NET Entity Framework uses XML Based Data Definition Language called Schema Definition Language (SDL) to define the EDM Schema.

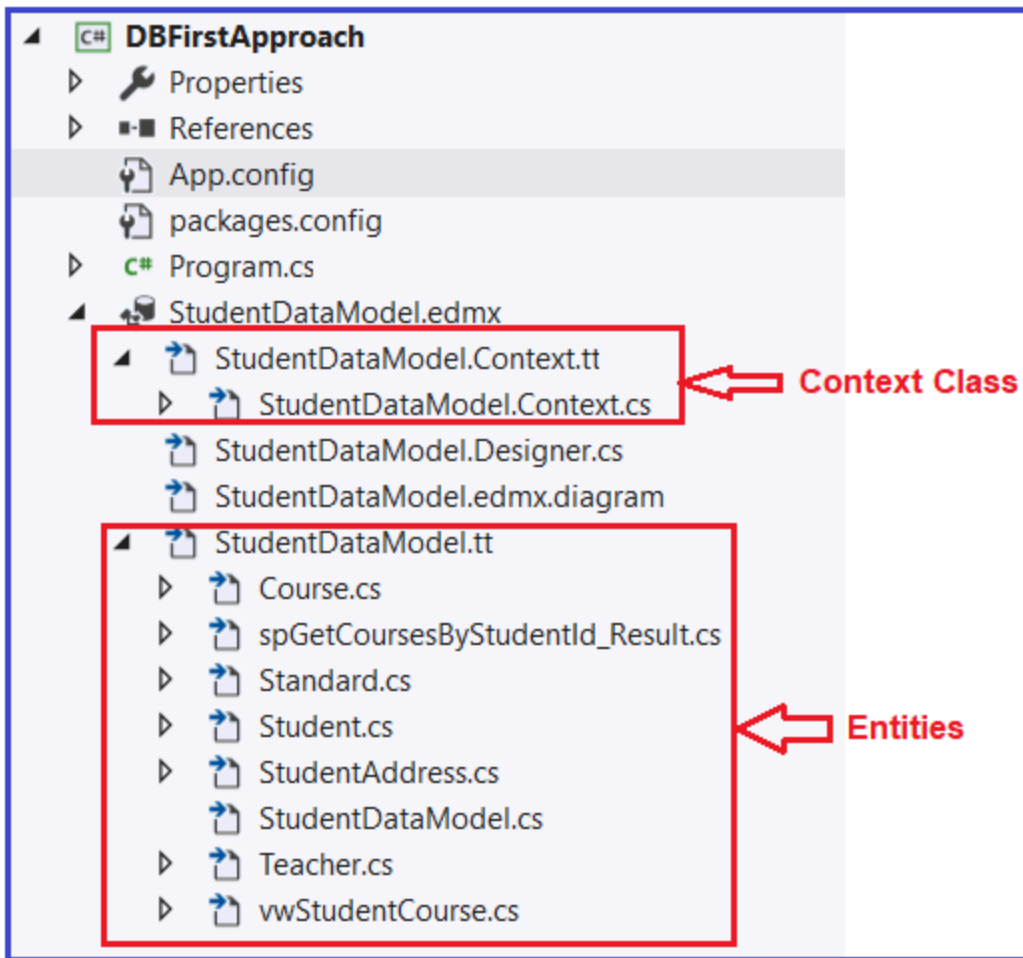
1. The SDL defines the Simple Types similarly to other Primitive Types such as String, Int32, Double, Decimal, and DateTime, etc.
2. An Enumeration, which defines a map of primitive values and names, is also considered a simple type. Enumerations are supported from framework version 5.0 onwards only. For example, you have a table for Gender, and its values are Male, Female, and Other. For this, the entity framework will create an enum.
3. Complex Types are created from an aggregation of other types. For example, a stored procedure returns a result set and that result set is going to be represented by a complex type.

The Entity Data Model primarily has three key components to describe the data structure or schema. They are as follows:

1. **Entity Type:** It represents the Database Table and Model Classes.
2. **Association type:** It represents the Foreign Keys of a database table and the Navigational Properties of a Model class.
3. **Property:** It represents the column names in a database table and the property names in a model class or Entities.

Context and Entity Classes

Every Entity Data Model (EDM) generates one context class for the database and an entity class for each database table and view. In order to verify this, expand the .edmx file in the solution explorer and open two important files i.e. <EDM Name>.Context.tt and <EDM Name>.tt, as shown in the below image.



StudentDataModel.Context.tt:

This T4 template file generates a context class whenever you change the Entity Data Model (.edmx file). You can see the context class file by expanding **StudentDataModel.Context.tt**. The context class resides in the <EDM Name>.context.cs file. In our case the context class file name is **StudentDataModel.Context.cs**. The default context class name is <DB Name>Entities. For example, the context class name for EF_Demo_DB is EF_Demo_DBEntities and is derived from the DbContext class as shown below.

```
namespace DBFirstApproach
{
    using System;
    using System.Data.Entity;
    using System.Data.Entity.Infrastructure;
    using System.Data.Entity.Core.Objects;
    using System.Linq;

    public partial class EF_Demo_DBEntities : DbContext
    {
        public EF_Demo_DBEntities()
            : base("name=EF_Demo_DBEntities")
        {
        }

        protected override void OnModelCreating(DbModelBuilder modelBuilder)
        {
        }
    }
}
```

```

        throw new UnintentionalCodeFirstException();
    }

    public virtual DbSet<Course> Courses { get; set; }
    public virtual DbSet<Standard> Standards { get; set; }
    public virtual DbSet<Student> Students { get; set; }
    public virtual DbSet<StudentAddress> StudentAddresses { get; set; }
    public virtual DbSet<Teacher> Teachers { get; set; }
    public virtual DbSet<vwStudentCourse> vwStudentCourses { get; set; }

    public virtual int spDeleteStudent(Nullable<int> studentId)
    {
        var studentIdParameter = studentId.HasValue ?
            new ObjectParameter("StudentId", studentId) :
            new ObjectParameter("StudentId", typeof(int));

        return
            ((IObjectContextAdapter)this).ObjectContext.ExecuteFunction("spDeleteStudent",
                studentIdParameter);
    }

    public virtual ObjectResult<spGetCoursesByStudentId_Result>
    spGetCoursesByStudentId(Nullable<int> studentID)
    {
        var studentIDParameter = studentID.HasValue ?
            new ObjectParameter("StudentID", studentID) :
            new ObjectParameter("StudentID", typeof(int));

        return
            ((IObjectContextAdapter)this).ObjectContext.ExecuteFunction<spGetCoursesByStu
            ("spGetCoursesByStudentId", studentIDParameter);
    }

    public virtual ObjectResult<Nullable<decimal>> spInsertStudent(Nullable<int>
    standardId, string firstName, string lastName)
    {
        var standardIdParameter = standardId.HasValue ?
            new ObjectParameter("StandardId", standardId) :
            new ObjectParameter("StandardId", typeof(int));

        var firstNameParameter = firstName != null ?
            new ObjectParameter("FirstName", firstName) :
            new ObjectParameter("FirstName", typeof(string));

        var lastNameParameter = lastName != null ?
            new ObjectParameter("LastName", lastName) :
            new ObjectParameter("LastName", typeof(string));

        return
            ((IObjectContextAdapter)this).ObjectContext.ExecuteFunction<Nullable<decimal>>
            ("spInsertStudent", standardIdParameter, firstNameParameter, lastNameParameter);
    }

```

```

    }

    public virtual int spUpdateStudent(Nullable<int> studentId, Nullable<int>
standardId, string firstName, string lastName)
    {
        var studentIdParameter = studentId.HasValue ?
            new ObjectParameter("StudentId", studentId) :
            new ObjectParameter("StudentId", typeof(int));

        var standardIdParameter = standardId.HasValue ?
            new ObjectParameter("StandardId", standardId) :
            new ObjectParameter("StandardId", typeof(int));

        var firstNameParameter = firstName != null ?
            new ObjectParameter("FirstName", firstName) :
            new ObjectParameter("FirstName", typeof(string));

        var lastNameParameter = lastName != null ?
            new ObjectParameter("LastName", lastName) :
            new ObjectParameter("LastName", typeof(string));

        return
            ((IObjectContextAdapter)this).ObjectContext.ExecuteFunction("spUpdateStudent",
studentIdParameter, standardIdParameter, firstNameParameter, lastNameParameter)
    }
}

```

StudentDataModel.tt:

The **StudentDataModel.tt** is a T4 template file that generates entity classes for each Database Table, View, Stored Procedure, and Function. Entity classes are POCO (Plain Old CLR Object) classes.

Entity Classes:

The following code snippet shows the Student entity.

```

namespace DBFirstApproach
{
    using System;
    using System.Collections.Generic;

    public partial class Student
    {
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
"CA2214:DoNotCallOverridableMethodsInConstructors")]
        public Student()
        {
            this.Courses = new HashSet<Course>();
        }
    }
}

```

```

        public int StudentId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public Nullable<int> StandardId { get; set; }

        public virtual Standard Standard { get; set; }
        public virtual StudentAddress StudentAddress { get; set; }
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
"CA2227:CollectionPropertiesShouldBeReadOnly")]
        public virtual ICollection<Course> Courses { get; set; }
    }
}

```

The following code snippet shows the StudentAddress entity.

```

namespace DBFirstApproach
{
    using System;
    using System.Collections.Generic;

    public partial class StudentAddress
    {
        public int StudentId { get; set; }
        public string Address1 { get; set; }
        public string Address2 { get; set; }
        public string Mobile { get; set; }
        public string Email { get; set; }

        public virtual Student Student { get; set; }
    }
}

```

The following code snippet shows the Course entity.

```

namespace DBFirstApproach
{
    using System;
    using System.Collections.Generic;

    public partial class Course
    {
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
"CA2214:DoNotCallOverridableMethodsInConstructors")]
        public Course()
        {
            this.Students = new HashSet<Student>();
        }

        public int CourseId { get; set; }
        public string CourseName { get; set; }
    }
}

```

```

        public Nullable<int> TeacherId { get; set; }

        public virtual Teacher Teacher { get; set; }
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
"CA2227:CollectionPropertiesShouldBeReadOnly")]
        public virtual ICollection<Student> Students { get; set; }
    }
}

```

You can check the other entities as well.

Database Views Classes:

The following code snippet shows the **vwStudentCourse**.

```

namespace DBFirstApproach
{
    using System;
    using System.Collections.Generic;

    public partial class vwStudentCourse
    {
        public int StudentId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public int CourseId { get; set; }
        public string CourseName { get; set; }
    }
}

```

Note: The point that you need to understand here is for each database table and view, EDM creates an entity.

In the next article, I am going to discuss the [Model Browser in Entity Framework](#). Here, in this article, I try to explain the **Entity Data Model (EDM) of the Entity Framework Database First Approach** and I hope you enjoyed this Entity Data Model (EDM) of Entity Framework Database First Approach article. Please give your valuable feedback and suggestions about this article.